Software Architecture Overview
Open Clinical Record

Open Clinical Record Internship Project

MVP Scope Revision — September 2026

Contents

1 Status

This document records the architectural direction for the reduced Open Clinical Record (OCR) internship MVP. The architecture is aligned with the revised SRS, current logical ERD, and project constraints to prioritize a deliverable system within the remaining one-month period.

2 Architectural Intent

The architecture should be simple, modular, maintainable, and practical for an internship-scale implementation. The MVP contains exactly three business modules: **Patient Management**, **Patient Chart**, and **Appointment Management**.

Authentication, authorization, validation, audit logging, and error handling are cross-cutting support concerns. They are not additional business modules.

The architecture separates the React user interface, ASP.NET Core API/application layer, business logic, and persistence so the three core workflows can be developed and tested independently while remaining connected through the patient record.

3 Technology Stack

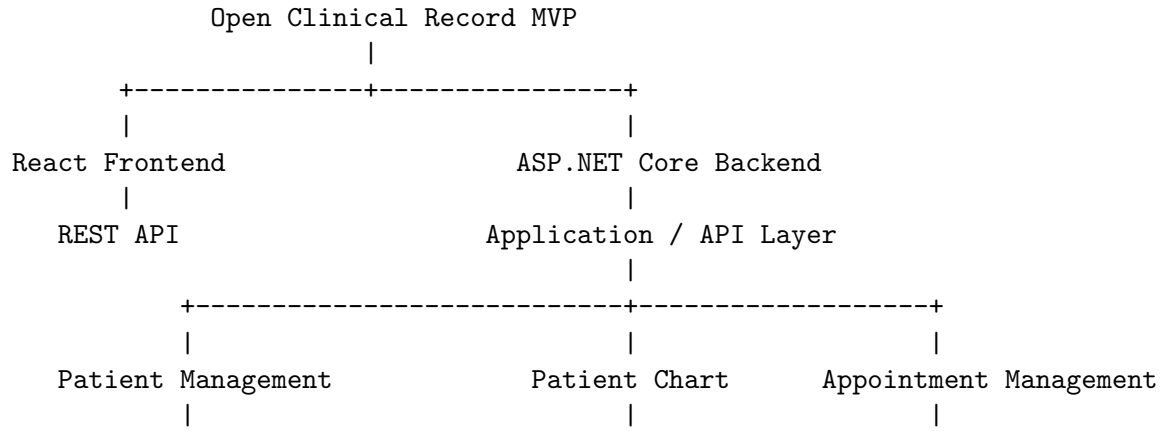
The approved implementation stack is:

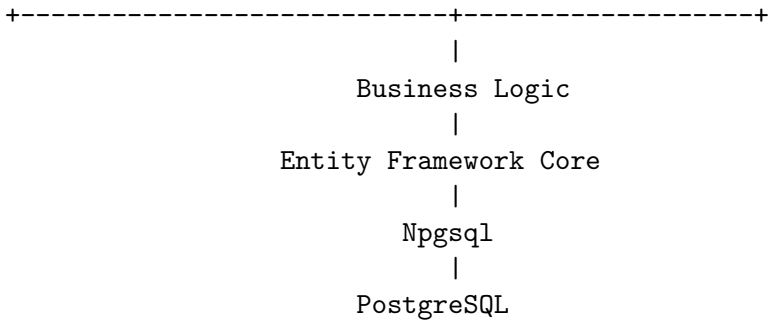
- **Frontend:** React for the web user interface.
- **Backend:** .NET with ASP.NET Core.
- **API style:** RESTful HTTP API between React and ASP.NET Core.
- **Database:** PostgreSQL.
- **Data access:** Entity Framework Core with the official PostgreSQL provider (Npgsql), unless a justified project decision changes the persistence approach.

PostgreSQL is the selected database engine for the internship MVP and shall be used consistently in the database design, local development environment, application configuration, migrations, testing, and deployment documentation.

International healthcare standards and FHIR integration are **not MVP requirements**. The architecture should avoid unnecessary complexity while leaving reasonable room for future extension.

4 Initial Logical View





The primary request flow is:

```
React UI -> REST API -> ASP.NET Core -> Business Logic -> EF Core/Npgsql -> PostgreSQL
```

The backend is responsible for validation, authorization, business rules, and data integrity. The frontend must not be treated as the security boundary.

5 Core Module Boundaries

5.1 Patient Management

This module manages the basic patient lifecycle required by the MVP: registration, unique patient identification, search, profile viewing, permitted demographic/contact updates, and basic patient status.

5.2 Patient Chart

This module provides the longitudinal patient context required by the MVP: demographics, status, allergies, relevant medication/history information, important alerts where required, and appointment/visit history.

The chart is intentionally limited. Full clinical encounter documentation, diagnosis, treatment, prescriptions, and advanced medical reports are deferred.

5.3 Appointment Management

This module manages the planned and basic operational visit workflow: appointment creation, viewing, rescheduling, cancellation, status, check-in, and basic walk-in support.

Appointments must remain linked to the correct patient. A walk-in may create a visit without a fabricated prior appointment.

6 Cross-Cutting Concerns

The application has exactly three application roles:

- Receptionist / Front Desk
- Nurse / Clinical Staff
- Clinician / Doctor

Authentication, role-based authorization, validation, error handling, and basic audit logging support the three modules. Access to a patient chart does not automatically grant permission to modify every piece of patient information.

7 API Direction

React communicates with ASP.NET Core through RESTful HTTP endpoints. The API should expose clear domain operations for patients, charts, appointments, and visits rather than unrestricted database CRUD.

For the internship MVP, API design should prioritize the endpoints needed for the three core workflows. External interoperability APIs and FHIR resources are deferred.

8 Data and Persistence Direction

The current logical data model is defined in `docs/03-requirements/ER/OCR_MVP_ERD.html` and summarized in `docs/05-data/README.md`. The model is intentionally small and normalized enough to support the MVP without introducing unnecessary enterprise entities.

PostgreSQL is the authoritative persistence engine for the MVP. Database-specific implementation should use PostgreSQL-compatible types, constraints, indexes, migrations, and transaction behavior. Application code should access PostgreSQL through the persistence layer rather than embedding database-specific SQL throughout business logic.

The current logical model includes:

- Patient
- Allergy
- Medication History / relevant medication information
- Patient Alert
- Appointment
- Appointment History/Event
- Visit
- User
- Audit Event

The central relationship is:

```
Patient
  |
  +---- Appointments
  |         |
  |         +---- Visit / Check-in
  |
  +---- Patient Chart information
```

A visit may originate from an appointment or from a walk-in. Appointment history must not be silently lost when an appointment is rescheduled or cancelled.

9 Security and Data Integrity

The backend shall enforce authorization and validate all important patient and appointment operations. Patient information must not be exposed to unauthorized users.

PostgreSQL constraints and application-level validation should work together to protect referential integrity, uniqueness, required relationships, and valid status transitions. The system

shall prevent duplicate patient identifiers, invalid relationships, and unsafe appointment operations. Failed operations shall not falsely report success or leave incomplete records where the application can prevent this.

10 Deferred Functionality

The following capabilities are explicitly outside the committed MVP:

- Full clinical encounter documentation
- Diagnosis and treatment documentation
- Prescription management
- Advanced medical reports and document amendment workflows
- Laboratory, pharmacy, billing, insurance, and radiology
- Patient portal/mobile application
- SMS and external integrations
- FHIR or other international standards implementation
- Advanced analytics and enterprise scheduling
- AI clinical decision support

These features may be considered only after the three core modules are completed, tested, and demonstrated, and only if sufficient time remains.

11 Architecture Decisions

The current baseline decisions are:

- React is the frontend technology.
- ASP.NET Core/.NET is the backend technology.
- REST is the primary frontend-to-backend API style.
- PostgreSQL is the selected MVP database engine.
- Entity Framework Core with Npgsql is the planned persistence approach.
- The system is organized around Patient Management, Patient Chart, and Appointment Management.
- The logical ERD is maintained as the current MVP data-model baseline.
- International standards/FHIR are deferred from the internship MVP.

12 Remaining Decisions

The following remain project decisions rather than database-engine selection decisions:

- Exact REST resources/endpoints.
- Authentication implementation and session/token approach.
- Final role-permission matrix after clinical confirmation.
- Deployment approach appropriate to the internship environment.
- PostgreSQL deployment configuration and backup procedure.

The logical ERD and three-module data-model baseline are already established and should not be expanded unless a justified MVP requirement requires a change.

13 Implementation Priority

Architecture and project work shall follow this priority:

1. Patient Management
2. Patient Chart
3. Appointment Management
4. Required authentication, authorization, validation, and error handling
5. Testing and bug fixing
6. Optional future functionality only if the MVP is complete